candidates built daily. And every release candidate can potentially be promoted to production. Will we have to update the semantic version for each new code commit, with a different approach based on its content (major, minor, patch)? The path from code commit to production should be automated as much as possible, trying to eliminate manual intervention. If we go with continuous deployment, even the promotion to production will happen automatically. What should we do?

One solution would be using the Git commit hash to version release candidates—that would be automated, traceable, and reliable, and you wouldn't need Git tags. You could use the commit hash as is (for example, `486105e261cb346b87920aaa4ea6dce6eebd6223`) or use it as the base for generating a more human-friendly number. For example, you could prefix it with a timestamp or with an increasing sequence number, with the goal of making it possible to tell which release candidate is the newest (for example, `20220731210356-486105e261cb346b87920aaa4ea6dce6eebd6223`).

Still, semantic versioning and similar strategies have their place in continuous delivery. They can be used as *display names* in addition to the unique identifier, as Dave Farley suggests in his book *Continuous Delivery Pipelines* (2021). That would be a way to provide users with information about the release candidate while still making it possible to benefit from continuous delivery.

For Polar Bookshop, we'll adopt a simple solution and use the Git commit hash directly to identify our release candidates. Therefore, we'll ignore the version number configured in the Gradle project (which could instead be used as the display version name). For example, a release candidate for Catalog Service would be `ghcr.io/`